1. **Aim/Overview of the practical:** Create employee salary program using NodeJS.
2. **Objective of the practical:** To create a Node.js-based program that manages employee salary records through basic CRUD operations.

3. **Code for experiment/practical:**
   EmployeeForm.jsx

```
import React, { useState, useEffect } from "react";

import axios from "axios";
import "../styles/EmployeeForm.css";

const EmployeeForm = ({ fetchEmployees, editEmployee, setEditEmployee }) => {
  const [formData, setFormData] = useState({
    name: "",
    hoursWorked: "",
    hourlyRate: "",
  });

  useEffect(() => {
    if (editEmployee && editEmployee.name) {
      setFormData({
        name: editEmployee.name || "",
        hoursWorked: editEmployee.hoursWorked || "",
        hourlyRate: editEmployee.hourlyRate || "",
      });
    }
  }, [editEmployee]);
```

```
  const handleChange = (e) =>
    setFormData({ ...formData, [e.target.name]: e.target.value });

  const handleSubmit = async (e) => {
    e.preventDefault();
    const salary = formData.hoursWorked * formData.hourlyRate;

    if (editEmployee) {
      await axios.put(`http://localhost:5000/api/employees/${editEmployee._id}`, {
        ...formData,
        salary,
      });
      setEditEmployee(null); // reset
    } else {
      await axios.post("http://localhost:5000/api/employees", {
        ...formData,
        salary,
      });
    }

    setFormData({ name: "", hoursWorked: "", hourlyRate: "" });
    fetchEmployees();
  };

  return (
    <div className="form-container">
      <h2>{editEmployee ? "Edit Employee" : "Add Employee"}</h2>
      <form onSubmit={handleSubmit}>
        <input type="text" name="name" placeholder="Employee Name"
value={formData.name} onChange={handleChange} required />
        <input type="number" name="hoursWorked" placeholder="Hours Worked"
value={formData.hoursWorked} onChange={handleChange} required />
        <input type="number" name="hourlyRate" placeholder="Hourly Rate (₹)"
value={formData.hourlyRate} onChange={handleChange} required />
        <button type="submit">{editEmployee ? "Update" : "Add"}</button>
      </form>
    </div>
  );
};

export default EmployeeForm;
```

Models

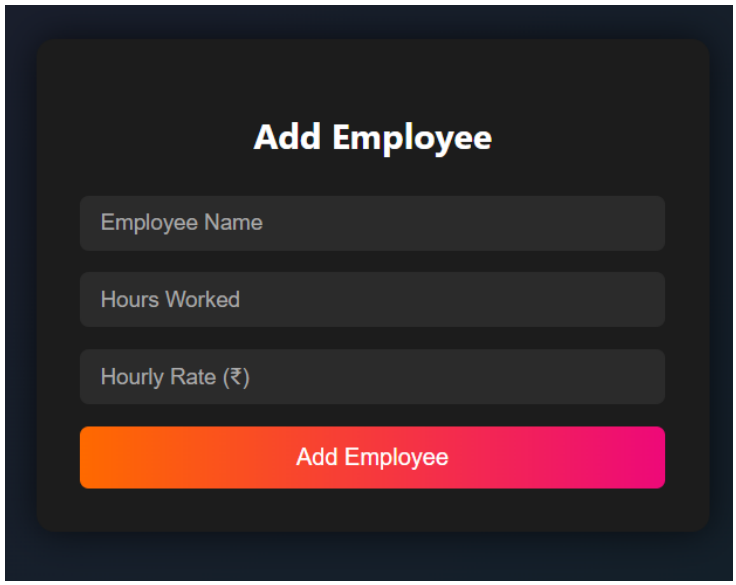
```
const mongoose = require("mongoose");
const employeeSchema = new mongoose.Schema({
  name: String,
  hoursWorked: Number,
  hourlyRate: Number,
  salary: Number,
});
module.exports = mongoose.model("Employee", employeeSchema);
```

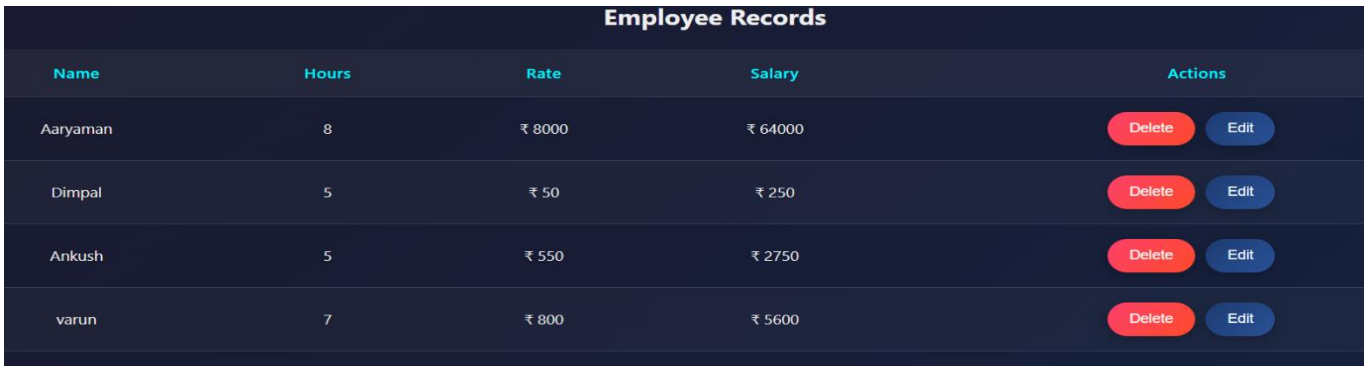
Server.js

```
const express = require("express");
const mongoose = require("mongoose");
const cors = require("cors");

const Employee = require("./models/Employee");
const app = express();
app.use(cors());
app.use(express.json());
mongoose
  .connect("mongodb://localhost:27017/employees", {
    useNewUrlParser: true,
    useUnifiedTopology: true,
  })
  .then(() => console.log("MongoDB connected"))
  .catch((err) => console.log("MongoDB Error:", err));

// Routes
app.get("/api/employees", async (req, res) => {
  const employees = await Employee.find();
  res.json(employees);
});
app.post("/api/employees", async (req, res) => {
  const newEmp = await Employee.create(req.body);
  res.json(newEmp);
});
app.put("/api/employees/:id", async (req, res) => {
  const updatedEmp = await Employee.findByIdAndUpdate(req.params.id, req.body, {
new: true });
  res.json(updatedEmp);
});
```

```
app.delete("/api/employees/:id", async (req, res) => {
  await Employee.findByIdAndDelete(req.params.id);
  res.json({ message: "Deleted" });
});
app.listen(5000, () => console.log("Server running on port 5000"));
```

4. Output:





5. Learning Outcomes (What I have learnt):

1. Understand how to build REST APIs using Node.js and Express.js.
2. Learn to implement salary data handling using JavaScript and JSON.
3. Gain hands-on experience in developing backend logic for employee management systems.